03 — Denormalization: When & Why

Table of Contents

Learning Objectives

After completing this section you will be able to:

- Explain the specific trade-offs of denormalization
- Choose the right denormalization technique for a given problem
- Use PostgreSQL materialized views as safe denormalization
- Implement counter caches and summary tables
- Avoid the common pitfalls of denormalization (stale data, update complexity)

What is Denormalization?

Denormalization is the deliberate introduction of redundancy into a normalized schema in order to improve read performance.

It is a **conscious, justified trade-off**:

```
Denormalization buys:     read performance (fewer JOINs, smaller result sets)
Denormalization costs:    write complexity, storage space, risk of inconsistency
```

Denormalization is **not** the same as bad design. It is a performance optimization applied to an already-correct schema, when measurements show the normalized schema is too slow.

Key principle: Normalize first. Then denormalize where profiling shows it is necessary.

Normalization vs. Denormalization Trade-offs

```
                    NORMALIZED                      DENORMALIZED
                    ──────────────────────────────────────────────
Write overhead      Low (one place to update)       High (multiple places)
Read overhead       Higher (JOINs needed)            Lower (data pre-joined)
Storage             Minimal                          Larger (redundancy)
Consistency risk    None                             Inconsistency possible
Query complexity    JOIN-heavy                       Simpler SELECTs
Cache efficiency    Smaller tables, better cache     Wider rows, more cache miss
Update anomalies    None                             Possible if not managed

Best for:           OLTP (many writes, reads         OLAP (rare writes,
                    on normalized data)              many complex reads)
```

Denormalization Techniques

Technique 1: Add redundant column (copy a frequently joined value)

```sql
-- Normalized:
SELECT o.order_id, u.email
FROM orders o JOIN users u ON u.user_id = o.user_id
WHERE o.order_id = 12345;

-- Denormalized: add user_email directly to orders
ALTER TABLE orders ADD COLUMN user_email TEXT;

-- Pros: no JOIN needed
-- Cons: if user changes email, must update orders too (or it goes stale)
-- Decision: acceptable if historical email-at-order-time is the right semantic
```

Technique 2: Pre-join tables

```sql
-- Normalized order details view:
CREATE VIEW order_line_items AS
SELECT
    o.order_id, o.created_at, o.status,
    u.full_name, u.email,
    oi.product_id, p.name AS product_name, oi.quantity, oi.unit_price
FROM orders o
JOIN users u ON u.user_id = o.user_id
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id;

-- Denormalized: materialized for heavy reporting use
CREATE MATERIALIZED VIEW order_line_items_mat AS
SELECT ... (same as above);
```

Technique 3: Stored aggregates (counter caches)

```sql
-- Normalized (requires COUNT query every time):
SELECT count(*) FROM posts WHERE user_id = 42;

-- Denormalized: store the count directly on users
ALTER TABLE users ADD COLUMN post_count INTEGER NOT NULL DEFAULT 0;
-- Must be maintained by triggers or application code!
```

Technique 4: Flattened/wide table for analytics

```sql
-- Star schema (data warehouse style):
-- One wide "fact" table with all pre-joined dimension values
CREATE TABLE sales_facts (
    sale_id         BIGINT  PRIMARY KEY,
    sale_date       DATE,
    -- denormalized customer dimensions
    customer_id     INTEGER,
    customer_name   TEXT,
    customer_region TEXT,
    customer_segment TEXT,
    -- denormalized product dimensions
    product_id      INTEGER,
    product_name    TEXT,
    category_name   TEXT,
    -- denormalized time dimensions
    year            SMALLINT,
    quarter         SMALLINT,
    month           SMALLINT,
    day_of_week     SMALLINT,
    -- measures
    quantity        INTEGER,
    unit_price      NUMERIC(10,4),
    discount        NUMERIC(5,4),
    total_amount    NUMERIC(12,4)
);
```

When to Denormalize

Signs that denormalization may be needed

1. **Profiler shows specific JOINs as bottlenecks** on hot queries
2. **Reporting queries** run for minutes that must run in seconds
3. **Read-heavy tables** (>10:1 read:write ratio)
4. **Dashboards** that aggregate the same data repeatedly
5. **Historical snapshots** needed (price-at-order-time, address-at-order-time)

Signs that denormalization is premature

1. You haven't profiled yet — "this will be slow"
2. Write frequency is high (e-commerce cart, real-time feeds)
3. The "slow" query hasn't been indexed yet

4. Read throughput is actually fine but you're guessing

```
Decision framework:
  1. Profile: identify the slow queries
  2. Index: add appropriate indexes
  3. View: use views for query simplification
  4. Materialize: use materialized views for pre-computed results
  5. Denormalize: as a last resort, add redundant columns
```

Materialized Views as Managed Denormalization

Materialized views are PostgreSQL's recommended first step for denormalization — they provide the read performance of a denormalized table while making the "refresh" responsibility explicit.

```sql
-- Create materialized view for a heavy reporting query
CREATE MATERIALIZED VIEW monthly_sales_summary AS
SELECT
    date_trunc('month', o.created_at)::DATE AS month,
    p.category_id,
    cat.name AS category_name,
    count(DISTINCT o.order_id) AS order_count,
    sum(oi.quantity) AS units_sold,
    sum(oi.quantity * oi.unit_price) AS gross_revenue,
    sum(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS net_revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p     ON p.product_id = oi.product_id
JOIN categories cat ON cat.category_id = p.category_id
WHERE o.status IN ('paid', 'shipped', 'delivered')
GROUP BY 1, 2, 3
WITH DATA;

-- Add indexes to the materialized view
CREATE INDEX idx_monthly_sales_month ON monthly_sales_summary (month);
CREATE INDEX idx_monthly_sales_cat   ON monthly_sales_summary (category_id);

-- Refresh strategies:
-- Manual refresh (simplest)
REFRESH MATERIALIZED VIEW monthly_sales_summary;

-- Concurrent refresh (doesn't lock reads!)
REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_sales_summary;
-- (requires a unique index on the mat view)
CREATE UNIQUE INDEX ON monthly_sales_summary (month, category_id);

-- Scheduled refresh via pg_cron (every hour)
SELECT cron.schedule('refresh_monthly_sales', '0 * * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_sales_summary');
```

Derived and Computed Columns

Generated columns (PostgreSQL 12+)

```
-- Computed at write time, stored (persistent denormalization)
CREATE TABLE order_items (
    order_id    BIGINT        NOT NULL,
    product_id  INTEGER       NOT NULL,
    quantity    INTEGER       NOT NULL CHECK (quantity > 0),
    unit_price  NUMERIC(10,4) NOT NULL,
    discount    NUMERIC(5,4)  NOT NULL DEFAULT 0,
    -- Computed columns (always consistent, no trigger needed)
    line_total  NUMERIC(12,4) GENERATED ALWAYS AS
                    (quantity * unit_price * (1 - discount)) STORED,
    PRIMARY KEY (order_id, product_id)
);

-- Can be indexed:
CREATE INDEX idx_order_items_total ON order_items (line_total);
```

Counter Caches

A counter cache stores a pre-computed count to avoid `COUNT(*)` at read time.

```
-- Table with counter cache
CREATE TABLE users (
    user_id      BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email        TEXT    NOT NULL UNIQUE,
    -- Counter caches (denormalized for performance)
    post_count   INTEGER NOT NULL DEFAULT 0 CHECK (post_count >= 0),
    follower_count INTEGER NOT NULL DEFAULT 0 CHECK (follower_count >= 0),
    following_count INTEGER NOT NULL DEFAULT 0 CHECK (following_count >= 0)
);

-- Trigger to maintain post_count
CREATE OR REPLACE FUNCTION update_user_post_count()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE users SET post_count = post_count + 1 WHERE user_id = NEW.user_id;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE users SET post_count = post_count - 1 WHERE user_id = OLD.user_id;
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_user_post_count
AFTER INSERT OR DELETE ON posts
```

```
FOR EACH ROW EXECUTE FUNCTION update_user_post_count();

-- Periodic reconciliation (safety net to fix drift)
CREATE OR REPLACE PROCEDURE reconcile_post_counts()
LANGUAGE SQL AS $$
    UPDATE users u
    SET post_count = (SELECT count(*) FROM posts p WHERE p.user_id = u.user_id);
$$;

-- Run reconciliation weekly
SELECT cron.schedule('reconcile_post_counts', '0 2 * * 0',
    'CALL reconcile_post_counts()');
```

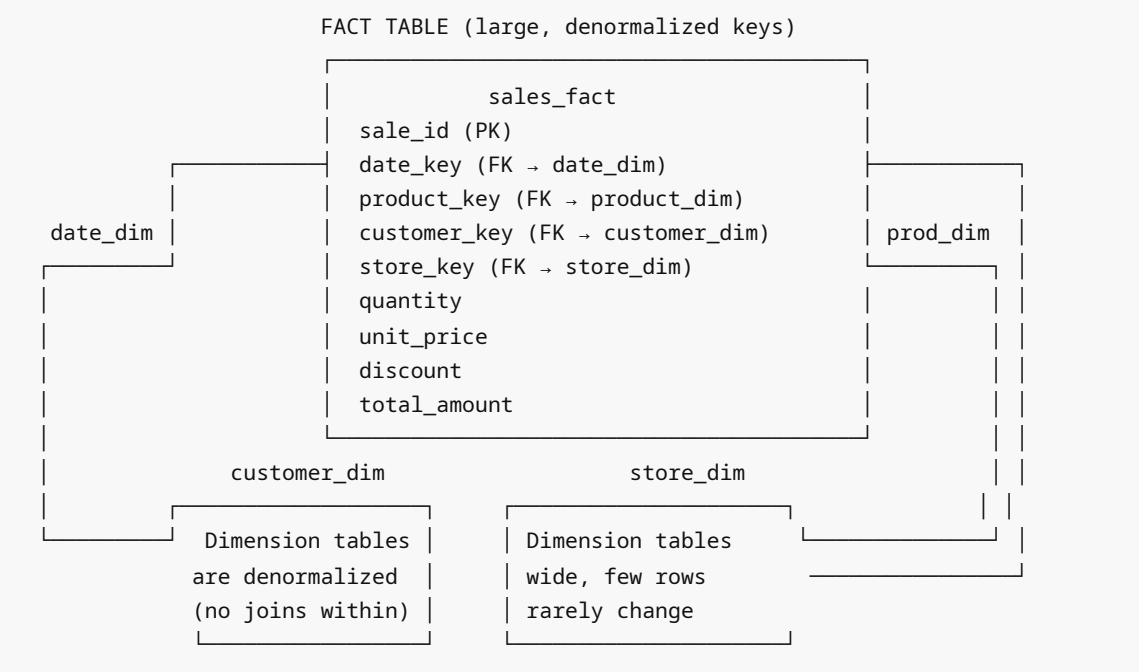
Pre-aggregated Summary Tables

```
-- Daily summary table (denormalized from raw events)
CREATE TABLE daily_user_activity (
    activity_date   DATE    NOT NULL,
    user_id         BIGINT  NOT NULL,
    page_views      INTEGER NOT NULL DEFAULT 0,
    sessions        INTEGER NOT NULL DEFAULT 0,
    purchases       INTEGER NOT NULL DEFAULT 0,
    revenue         NUMERIC(12,4) NOT NULL DEFAULT 0,
    PRIMARY KEY (activity_date, user_id)
);

-- Populate via scheduled job
CREATE OR REPLACE PROCEDURE refresh_daily_activity(p_date DATE DEFAULT CURRENT_DATE
- 1)
LANGUAGE SQL AS $$
    INSERT INTO daily_user_activity (activity_date, user_id, page_views, sessions,
purchases, revenue)
    SELECT
        p_date,
        user_id,
        count(*) FILTER (WHERE event_type = 'page_view'),
        count(DISTINCT session_id),
        count(*) FILTER (WHERE event_type = 'purchase'),
        sum(amount) FILTER (WHERE event_type = 'purchase')
    FROM raw_events
    WHERE event_date = p_date
    GROUP BY user_id
    ON CONFLICT (activity_date, user_id) DO UPDATE SET
        page_views = EXCLUDED.page_views,
        sessions   = EXCLUDED.sessions,
        purchases  = EXCLUDED.purchases,
        revenue    = EXCLUDED.revenue;
$$;
```

Data Warehouse / OLAP Denormalization

Star schema




```sql
-- Star schema example
CREATE TABLE date_dim (
    date_key    INTEGER PRIMARY KEY,  -- YYYYMMDD
    full_date   DATE NOT NULL,
    year        SMALLINT NOT NULL,
    quarter     SMALLINT NOT NULL,
    month       SMALLINT NOT NULL,
    month_name  TEXT     NOT NULL,
    week        SMALLINT NOT NULL,
    day_of_week SMALLINT NOT NULL,
    day_name    TEXT     NOT NULL,
    is_weekend  BOOLEAN  NOT NULL,
    is_holiday  BOOLEAN  NOT NULL DEFAULT FALSE
);

CREATE TABLE product_dim (
    product_key     INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_id      INTEGER NOT NULL,  -- operational system ID
    product_name    TEXT    NOT NULL,
    category_name   TEXT    NOT NULL,  -- denormalized from categories table
    subcategory_name TEXT,
    brand_name      TEXT,
    unit_cost       NUMERIC(10,4),
    -- Slowly Changing Dimension type 2: effective dates
    effective_from  DATE    NOT NULL,
    effective_to    DATE,
```

```
    is_current      BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE sales_fact (
    sale_key        BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    date_key        INTEGER NOT NULL REFERENCES date_dim(date_key),
    product_key     INTEGER NOT NULL REFERENCES product_dim(product_key),
    customer_key    INTEGER NOT NULL,
    store_key       INTEGER NOT NULL,
    quantity        INTEGER NOT NULL,
    unit_price      NUMERIC(10,4) NOT NULL,
    discount_amount NUMERIC(10,4) NOT NULL DEFAULT 0,
    total_amount    NUMERIC(12,4) NOT NULL
);

-- Analytical query on star schema (no JOINs within dims needed)
SELECT
    d.year,
    d.quarter,
    p.category_name,
    sum(f.total_amount) AS revenue,
    sum(f.quantity)     AS units
FROM sales_fact f
JOIN date_dim    d ON d.date_key    = f.date_key
JOIN product_dim p ON p.product_key = f.product_key
WHERE d.year = 2024
  AND p.is_current = TRUE
GROUP BY 1, 2, 3
ORDER BY 1, 2, 3;
```

SQL Examples

Blog platform: normalized + strategic denormalization

```
-- Normalized base
CREATE TABLE posts (
    post_id         BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    author_id       BIGINT  NOT NULL REFERENCES users(user_id),
    title           TEXT    NOT NULL,
    body            TEXT    NOT NULL,
    status          TEXT    NOT NULL DEFAULT 'draft'
                        CHECK (status IN ('draft','published','archived')),
    published_at    TIMESTAMPTZ,
    -- Strategic denormalization #1: author info snapshot for listing queries
    author_name     TEXT    NOT NULL,  -- copied from users.full_name at publish
time
    -- Strategic denormalization #2: counter caches
    view_count      INTEGER NOT NULL DEFAULT 0,
    comment_count   INTEGER NOT NULL DEFAULT 0,
    like_count      INTEGER NOT NULL DEFAULT 0,
```

```
    -- Strategic denormalization #3: computed ranking score
    trending_score  REAL    GENERATED ALWAYS AS (
        (CASE status WHEN 'published' THEN 1 ELSE 0 END) *
        (COALESCE(view_count, 0) + COALESCE(comment_count * 5, 0) +
COALESCE(like_count * 3, 0))
    ) STORED,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Index on trending score for homepage feed
CREATE INDEX idx_posts_trending ON posts (trending_score DESC) WHERE status =
'published';

-- Fast "recent published posts by author" query (no JOIN needed for listing):
SELECT post_id, title, author_name, view_count, comment_count, like_count
FROM posts
WHERE status = 'published'
ORDER BY published_at DESC
LIMIT 20;
```

Common Mistakes

1. **Denormalizing before profiling** — This is premature optimization. First profile, then optimize.
2. **Not maintaining counters transactionally**

   ```
   -- BAD: race condition — two concurrent updates can both read 5 and set 6
   UPDATE users SET post_count = (SELECT count(*) FROM posts WHERE user_id = 1);
   -- GOOD: atomic increment
   UPDATE users SET post_count = post_count + 1 WHERE user_id = 1;
   ```

3. **Forgetting to refresh materialized views** — Stale materialized views are invisible problems. Always schedule refreshes and monitor staleness.
4. **Denormalizing columns that change frequently** — Denormalizing `users.city` into `orders` is fine (historical snapshot). Denormalizing `users.subscription_status` into every row is dangerous — status changes often.
5. **No reconciliation job for counter caches** — Triggers can be missed (bulk operations bypass triggers by default). Always have a periodic reconciliation.

Best Practices

1. **Profile first** — only denormalize proven bottlenecks.
2. **Try materialized views first** — they give most of the benefit with less risk.
3. **Document every denormalization** with a comment explaining what it optimizes and how consistency is maintained.
4. **Use triggers or application-level transactions** to maintain consistency.
5. **Implement reconciliation jobs** for all counter caches.

6. **Consider generated columns** for deterministic derived values — they're always consistent.
7. **Prefer materialized snapshots** over live denormalization for analytics use cases.

Performance Considerations

```
Denormalization impact summary:

Read improvement:
  Eliminating a JOIN on a 10M-row table:  ~10-100ms saved per query
  Eliminating a COUNT(*) with counter:    ~1-50ms saved per query
  Materialized view vs real-time:         100x-1000x faster for complex aggregates

Write overhead:
  Counter cache update (trigger):         +1-5ms per INSERT/DELETE
  Materialized view refresh (manual):     seconds to minutes for large views
  Concurrent mat view refresh:            background, no read lock

Storage overhead:
  Counter columns:                        ~4-8 bytes per row
  Copied text column (e.g., author_name): ~10-100 bytes per row
  Summary table:                          depends on cardinality
```

Interview Questions & Answers

Q1: When is denormalization appropriate?

A: Denormalization is appropriate when: (1) you have profiled and confirmed a specific query is too slow, (2) the query cannot be adequately optimized with indexes, (3) the table has a high read-to-write ratio, and (4) the consistency maintenance cost is acceptable. Never denormalize preemptively.

Q2: What is a materialized view and how is it different from a regular view?

A: A view is a stored query that executes on every access. A materialized view is a view whose result is physically stored as a table. Reads are extremely fast (no query execution). The trade-off is that the data can become stale and must be refreshed. `REFRESH MATERIALIZED VIEW CONCURRENTLY` refreshes without blocking reads (requires a unique index).

Q3: What is a counter cache and what are the risks?

A: A counter cache stores a pre-computed count (e.g., `post_count` in the users table) to avoid `COUNT(*)` at read time. Risks: (1) trigger failures can leave counts incorrect, (2) bulk operations (COPY, DELETE without WHERE) bypass row-level triggers, (3) concurrent updates can cause race conditions if not atomic. Mitigation: use atomic increments (`SET count = count + 1`), implement reconciliation jobs.

Q4: Is including `unit_price` in `order_items` an example of denormalization?

A: It depends on interpretation. Storing the price-at-order-time is intentional and semantically correct — it's a historical record, not redundancy. If you need "the current product price" you look at `products.price` . If you need "what the customer paid" you look at `order_items.unit_price` . These are different facts. This is "controlled temporal denormalization" — a standard practice, not an anomaly.

Q5: What is the difference between OLTP and OLAP normalization strategies?

A: OLTP (transactional) systems benefit from normalization — writes are frequent and should touch as few places as possible to maintain consistency. OLAP (analytical) systems benefit from denormalization — queries are complex, multi-table joins are expensive at scale, and writes are infrequent (batch ETL). Star/snowflake schemas are OLAP patterns that embrace denormalization.

Q6: How would you handle a denormalized counter that has drifted out of sync?

A: Run a reconciliation query: `UPDATE users SET post_count = (SELECT count(*) FROM posts WHERE posts.user_id = users.user_id)`. This is safe (idempotent) and should be run periodically (e.g., nightly) to correct any drift. Monitor the difference between expected and actual counts as a data quality metric.

Exercises with Solutions

Exercise 1

The following query runs every 100ms on a high-traffic homepage. Design an appropriate denormalization strategy.

```
SELECT p.post_id, p.title, u.full_name AS author,
       count(c.comment_id) AS comment_count,
       count(l.like_id)    AS like_count
FROM posts p
JOIN users u ON u.user_id = p.author_id
LEFT JOIN comments c ON c.post_id = p.post_id
LEFT JOIN likes l    ON l.post_id = p.post_id
WHERE p.status = 'published'
ORDER BY p.published_at DESC LIMIT 20;
```

Solution:

```
-- 1. Add counter caches to posts
ALTER TABLE posts
    ADD COLUMN comment_count INTEGER NOT NULL DEFAULT 0,
    ADD COLUMN like_count INTEGER NOT NULL DEFAULT 0,
    ADD COLUMN author_name TEXT NOT NULL DEFAULT '';

-- 2. Backfill current counts
UPDATE posts SET
    comment_count = (SELECT count(*) FROM comments c WHERE c.post_id =
posts.post_id),
    like_count    = (SELECT count(*) FROM likes l WHERE l.post_id = posts.post_id),
    author_name   = (SELECT full_name FROM users u WHERE u.user_id =
posts.author_id);

-- 3. Add indexes
CREATE INDEX idx_posts_published ON posts (published_at DESC) WHERE status =
'published';

-- 4. New query (no JOINs needed for the listing):
SELECT post_id, title, author_name, comment_count, like_count
FROM posts
```

```
WHERE status = 'published'
ORDER BY published_at DESC LIMIT 20;
```

Cross-References

- `01_normalization_1nf_2nf_3nf.md` — understand what you're denormalizing away from
- `10_design_patterns.md` — audit trails (a controlled form of denormalization)
- `../05_PostgreSQL_Core/05_json_jsonb.md` — JSONB as flexible denormalized storage
- `../07_Indexes/` — try indexes before denormalizing
- `../16_PostgreSQL_For_Data_Engineering/` — star schemas and data warehousing